

IBUKI Circle

One breath. One tap. Your circle responds. A multimodal recovery and prevention platform for people navigating substance use disorders — and the caregivers standing beside them.

The worst time to type

- Typing fails exactly when cognitive load peaks — a craving, a panic attack, a suspected overdose.
 - In that moment nobody needs another chatbot. They need one safe next action.
 - An emergency cannot wait for a model response — and a helpline number should never depend on model memory.
-

3 in 4 people with drug dependence in India receive no treatment.

National Survey on Extent and Pattern of Substance Use — MoSJE · AIIMS, 2019

**“AI personalizes the support.
Deterministic code controls safety.”**

— the one design rule behind every screen

01

One tap, one plan

From distress to a connected plan

One tap in

A tap, a spoken sentence, or a short message enters one validated endpoint — `/api/intervene`. There is no prompt to compose.

Risk checked first

The emergency macro and recognized danger requests bypass the model entirely; the deterministic router hands everything else to one of four specialists.

A connected plan

One structured-generation step — bounded retry, cross-provider fallback — returns coping steps, a breathing guide, an editable circle message, and verified helplines.

Multimodal, human-centred

Speak, listen, converse

Speech input, read-aloud, and a live speech-to-speech voice that keeps talking while it places breathing guides, support messages, and helplines directly on screen.

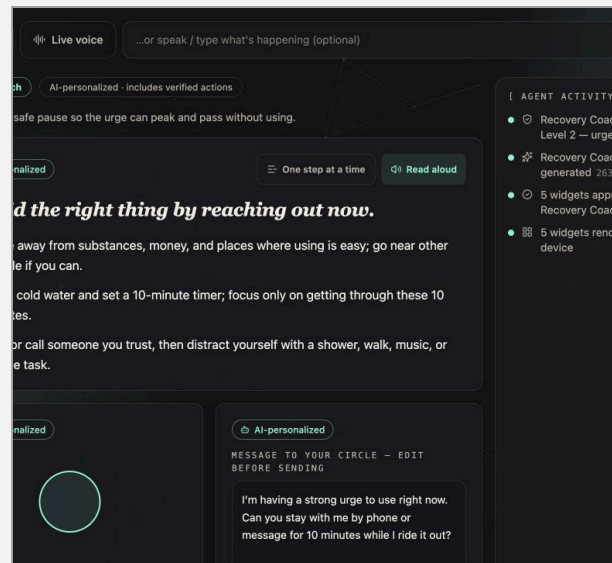
Eight languages, two modes

Plans generate live in English, Tamil, Hindi, Bengali, Telugu, Marathi, Kannada, and Malayalam. Caregiver mode returns what to say, what to avoid, and warning signs.

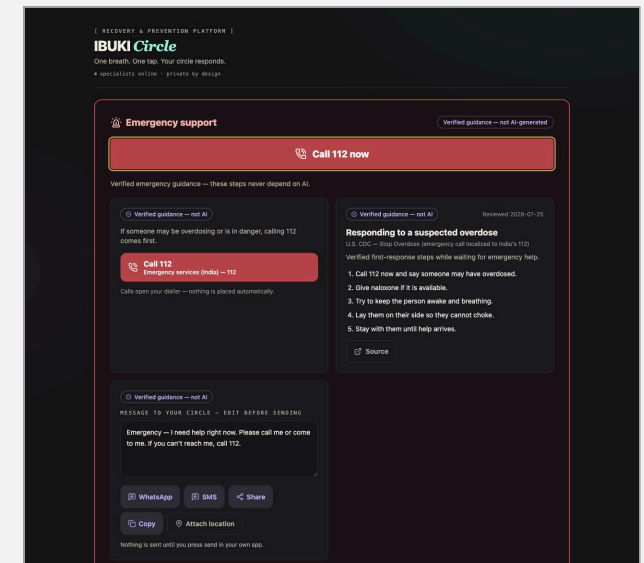
The judged application



Mobile home — one-tap situations



Live personalized intervention



Model-independent emergency protocol

REAL WORKING APPLICATION SHOWN DURING JUDGING

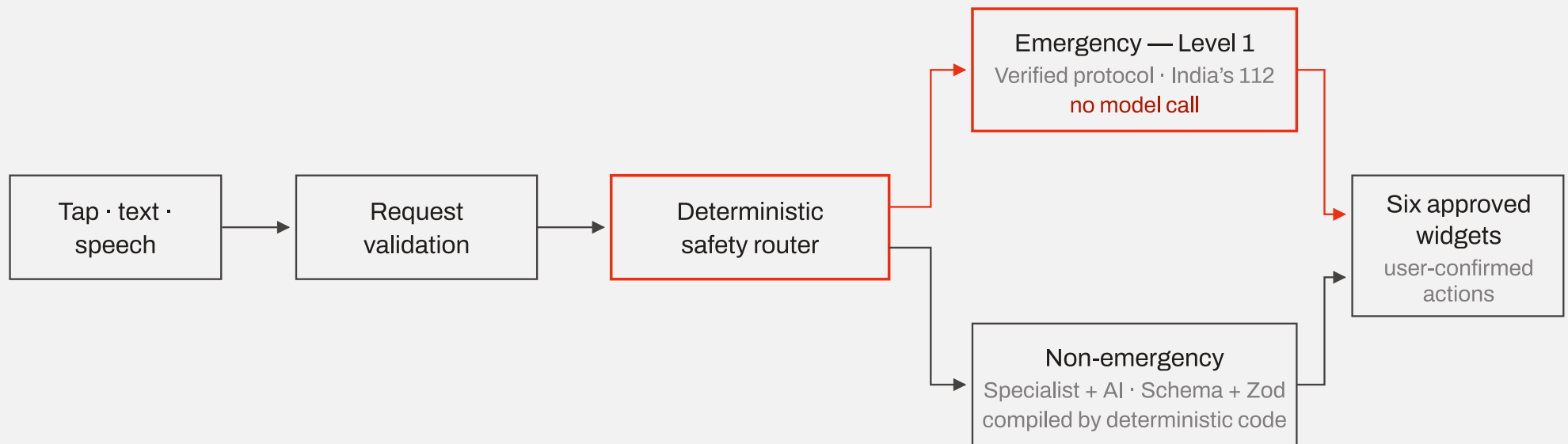
112

The emergency macro and recognized danger requests never wait for a model — deterministic code returns India's 112 action and reviewed response steps immediately. The model cannot lower the risk level, invent a helpline, or delay that path.

02

Under the hood

One pipeline, four specialists



AI writes structured intent. Deterministic code owns safety, UI, resources, and actions.

The stack

	LAYER	TECHNOLOGY	ROLE
01	Application	Next.js 16 App Router · React 19	UI, server routes, streaming
02	GenAI	GPT-5.6 Terra · Gemini 3.6 Flash · Realtime · TTS	Structured intent, live voice, read-aloud
03	Safety	TypeScript router · JSON Schema · Zod 4	Deterministic routing, strict contract
04	Tooling	Nx 23 · pnpm · Tailwind 4 · Vitest 4	Cached CI, 65 tests, Vercel + Docker

How we built it

Double validation

Strict JSON Schema at the API layer, Zod re-validation, then per-agent widget and resource allow-lists — invalid output is rejected, never substituted. 15 s hard timeouts; failover timing owned end-to-end.

CI quality gates

Frozen install → zero-warning lint → 65 tests across 13 files → production dependency audit → build → full-history secret scan. Emergency-phrases sweeps, schema contracts, and the rate limiter all covered.

Deploy-early discipline

13 production deploys in one day, every feature verified live before it was claimed. Localization shipped safely: 45 UI keys × 8 languages as tested constants; medical protocols stay English by design.

Contained voice, private core

Live voice, contained

OpenAI Realtime over WebRTC on a 10-minute ephemeral token — the server key never leaves. Five allow-listed screen tools, argument-validated and clamped; the emergency tool re-enters the same safety router.

Private by design

No database: transcripts, audio, location, contacts, and recovery history are never persisted. Server-side secrets, bounded input, no-store responses, CI dependency audit and secret scanning.

Stateless today, ready to scale

Stateless core

No application database or server-side session store — access is a stateless signed cookie. The standalone build runs identically on Vercel serverless and the node:22-alpine Docker image.

Efficient under load

One structured-generation step per non-emergency — bounded retry, cross-provider fallback, zero model latency on Level 1. Per-instance rate limits, schema-capped inputs, a 16 KB declared body limit.

The growth path

Distributed quotas on shared storage, multi-region deployment, country-specific registries, reviewed translations — and the provider boundary routing on availability, cost, and latency.

03

What's next

Future scope, in priority order

01 · Clinical validation

Professionally reviewed protocols, clinically validated Tamil and Hindi emergency translations, and a standing clinician advisory loop.

04 · Recovery journey

Non-punitive daily check-ins, a personal safety card, and engagement-based progress — never shame-based streaks.

02 · Production hardening

Distributed rate limiting, CSP, signed sessions, observability, and a deployed E2E and axe accessibility suite.

05 · Onboarding & persona

An optional, skippable profile — coping preferences, trusted contact, language — stored locally; the orchestrator already consumes every field.

03 · My Circle consent loop

A caregiver opens a short-lived link and taps “I’m on my way” — reflected live on the person’s screen. Upstash Redis, 24 h TTL, no PII.

06 · Moderated peer pilot

A peer-support pilot under human moderation, reporting, crisis escalation, abuse controls, audit trails, and clinical governance.

The submission, in numbers

3rd

Place — Second Runner-Up,
PromptWars Chennai 2026

65

tests passing across 13
test files

8

response languages,
generated live

12

one-tap situations across
two modes

4

specialist agents behind
one router

6

approved widget types — a
closed vocabulary

5

allow-listed live-voice
screen tools

4/4

CI gates green — lint, build,
audit, secret scan

Thank you

IBUKI Circle reduces the time between a moment of distress and a safe next action. 3rd Place · Second Runner-Up · PromptWars Chennai 2026. Source, architecture, screenshots, and this deck: github.com/tsathya98/promptwars-chennai-2026.